

Guía 2 | PlantUML | Casos de uso, clases y secuencia de análisis | FastKote

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Sistema de Generación de Cotizaciones FastKote para el negocio "Chichi está de Fiesta".
Operaciones / módulos seleccionados	Administrar Cotización, Administrar Cliente, Administrar Paquetes y Administrar catálogo de Insumos.
Referencia base	01_Guia_PlantUML_CU_Analisis_FastKote
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia en el proyecto FastKote.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso aplicada al proyecto FastKote. Su finalidad es mostrar cómo los casos de uso principales del Sistema de Generación de Cotizaciones pueden analizarse de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz ni en definir tecnologías de implementación, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos. Para esta propuesta se trabajan los casos de uso: Administrar Cotización, Administrar Cliente, Administrar Paquetes y Administrar catálogo de Insumos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación principal de FastKote.	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: Sistema de Generación de Cotizaciones FastKote

El caso se basa en el sistema FastKote, diseñado para apoyar el proceso de cotización de eventos del negocio "Chichi está de Fiesta". El sistema busca reducir errores manuales, centralizar información y permitir una gestión más ordenada de clientes, paquetes, cotizaciones e insumos de referencia.

Dato	Tipo sugerido	Descripción	Validación mínima
Cliente	Registro de contacto	Persona o entidad que solicita una cotización.	Nombre o razón social, cédula/RUC y teléfono obligatorios para vinculación comercial.
Cotización	Registro comercial	Documento o registro donde se asocian cliente, paquete, fecha del evento, detalle y total.	Debe mantener estado válido: Borrador, Enviada, Aceptada, Rechazada o Vencida.
Paquete	Oferta comercial	Agrupación de servicios/productos ofrecida como combo o propuesta base.	Nombre no duplicado, descripción y al menos un servicio/producto asociado.
Insumo	Catálogo básico	Material usado como referencia de costos para servicios o productos.	Nombre, marca y costo/precios registrados.

Acción / módulo principal	Caso de uso asociado	Descripción de análisis
Gestionar cotizaciones	Administrar Cotización	Crea, consulta, edita, actualiza estado, genera PDF y permite enviar cotizaciones al cliente.
Gestionar clientes	Administrar Cliente	Registra, consulta, actualiza o desactiva clientes para que sus datos estén disponibles en las cotizaciones.
Gestionar paquetes	Administrar Paquetes	Registra, consulta, edita o desactiva paquetes comerciales compuestos por servicios/productos.
Gestionar catálogo de insumos	Administrar catálogo de Insumos	Registra, consulta, actualiza o elimina insumos básicos usados como referencia de costos.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-02	El sistema debe permitir administrar cotizaciones de eventos, incluyendo creación, consulta, edición, actualización de estado, generación de PDF y envío por WhatsApp.	Administrador de Cotizaciones	Administrar Cotización
RF-03	El sistema debe permitir administrar clientes, registrando, consultando, actualizando o desactivando su información para vincularla a las cotizaciones.	Administrador de Cotizaciones	Administrar Cliente
RF-04	El sistema debe permitir administrar paquetes comerciales, registrando, consultando, editando o desactivando ofertas compuestas por servicios y productos.	Administrador del Sistema	Administrar Paquetes
RF-05	El sistema debe permitir administrar el catálogo básico de insumos, registrando, consultando, actualizando o eliminando insumos utilizados como referencia en el proceso de cotización.	Encargado de Inventario	Administrar catálogo de Insumos

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para los actores principales del sistema. En este caso, el Administrador de Cotizaciones interactúa con la administración de cotizaciones y clientes, el Administrador del Sistema con la administración de paquetes y el Encargado de Inventario con el catálogo básico de insumos.

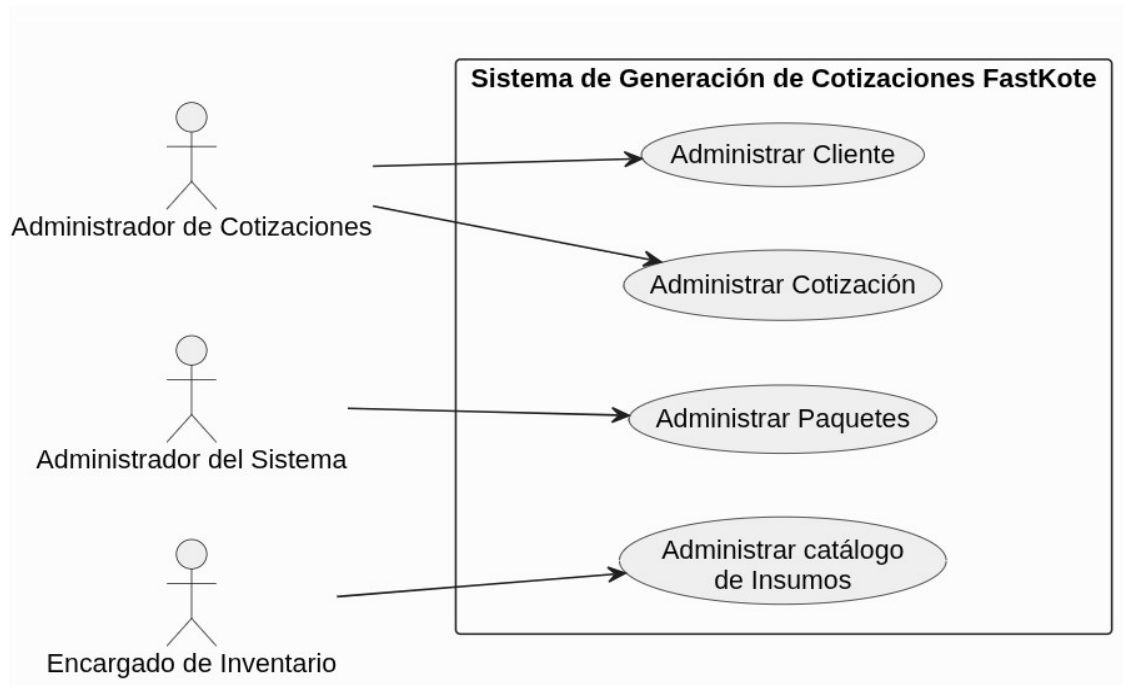
Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador de Cotizaciones" as AC
actor "Administrador del Sistema" as AS
actor "Encargado de Inventario" as EI

rectangle "Sistema de Generación de Cotizaciones FastKote" {
    usecase "Administrar Cotización" as UC1
    usecase "Administrar Cliente" as UC2
    usecase "Administrar Paquetes" as UC3
    usecase "Administrar catálogo\nde Insumos" as UC4
}

AC --> UC1
AC --> UC2
AS --> UC3
EI --> UC4
@enduml
```



Interpretación: el Administrador de Cotizaciones ejecuta las funciones centrales del proceso comercial, el Administrador del Sistema configura los paquetes de la oferta comercial y el Encargado de Inventario mantiene el catálogo básico de insumos como referencia de costos. El diagrama evita relaciones <<include>> o <<extend>> innecesarias en nivel 0 para mantener claridad académica.

6. Derivación de clases de análisis

A partir de los casos de uso seleccionados se identifican clases de análisis. Para mantener una separación clara de responsabilidades se utilizan los estereotipos boundary, control, entity y repository. Las clases boundary representan la interacción con el actor; las clases control coordinan la lógica del caso de uso; las entidades representan conceptos del dominio; y los repositorios representan la persistencia o colección de registros sin entrar en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con FastKote
Boundary / Frontera	Recibe datos del actor y muestra resultados.	FormularioCotizacion, FormularioCliente, FormularioPaquete, FormularioInsumo	Representan las vistas o formularios desde donde se ejecutan las operaciones principales.
Control	Coordina la lógica de cada caso de uso.	ControlCotizacion, ControlCliente, ControlPaquete, ControlInsumo	Validan datos, coordinan cálculos, consultas y persistencia.
Entity / Entidad	Representa información del dominio.	Cotizacion, DetalleCotizacion, Cliente, Evento, Paquete, ServicioProducto, Insumo	Contienen los datos principales manipulados por los casos de uso.
Repositorio / Colección	Conserva y recupera registros.	RepositorioCotizacion, RepositorioCliente, RepositorioPaquete, RepositorioInsumo	Permiten guardar, buscar, actualizar, eliminar o listar información de análisis.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

' =====
' BOUNDARY
' =====

class "FormularioCotizacion" <<boundary>> {
+ clickGenerarCotizacion()
}
```

```

+ clickConsultarCotizacion()
+ clickActualizarEstado()
+ mostrarResumen(cotizacion: Cotizacion)
+ mostrarMensaje(mensaje: String)
}

class "FormularioCliente" <<boundary>> {
+ clickRegistrarCliente()
+ clickConsultarCliente()
+ clickActualizarCliente()
+ mostrarCliente(cliente: Cliente)
+ mostrarMensaje(mensaje: String)
}

class "FormularioPaquete" <<boundary>> {
+ clickRegistrarPaquete()
+ clickConsultarPaquete()
+ clickEditarPaquete()
+ mostrarPaquete(paquete: Paquete)
+ mostrarMensaje(mensaje: String)
}

class "FormularioCatalogoInsumos" <<boundary>> {
+ clickRegistrarInsumo()
+ clickConsultarInsumo()
+ clickActualizarInsumo()
+ clickEliminarInsumo()
+ mostrarInsumo(insumo: Insumo)
+ mostrarMensaje(mensaje: String)
}

' =====
' CONTROL
' =====

class "ControlCotizacion" <<control>> {
+ generarCotizacion(clienteId: String, paqueteId: String, fechaEvento: Date): Resultado
+ consultarCotizacion(criterio: String): List<Cotizacion>
+ actualizarEstado(idCotizacion: String, estado: String): Resultado
+ calcularTotal(paquete: Paquete): double
}

class "ControlCliente" <<control>> {
+ registrarCliente(cedulaRuc: String, nombre: String, telefono: String, correo: String): Resultado
+ validarDatosCliente(cedulaRuc: String, nombre: String, telefono: String, correo: String): boolean
+ consultarCliente(criterio: String): List<Cliente>
+ actualizarCliente(cliente: Cliente): Resultado
+ desactivarCliente(idCliente: String): Resultado
}

class "ControlPaquete" <<control>> {
+ registrarPaquete(nombre: String, descripcion: String, items: List<ServicioProducto>): Resultado
+ validarDatosPaquete(nombre: String, descripcion: String, items: List<ServicioProducto>): boolean
+ consultarPaquete(criterio: String): List<Paquete>
+ editarPaquete(paquete: Paquete): Resultado
+ desactivarPaquete(idPaquete: String): Resultado
}

class "ControlCatalogoInsumos" <<control>> {
+ registrarInsumo(nombre: String, marca: String, costo: double): Resultado
+ validarDatosInsumo(nombre: String, marca: String, costo: double): boolean
+ consultarInsumo(criterio: String): List<Insumo>
+ actualizarInsumo(insumo: Insumo): Resultado
+ actualizarCosto(idInsumo: String, costo: double): Resultado
+ eliminarInsumo(idInsumo: String): Resultado
}

' =====
' ENTIDADES
' =====

class "Cliente" <<entity>> {
- idCliente: String
- cedulaRuc: String
- nombre: String
- telefono: String
- correo: String
- estado: String
}

class "Cotizacion" <<entity>> {
- idCotizacion: String
- fechaCotizacion: Date
- total: double
- estado: String
}

class "DetalleCotizacion" <<entity>> {

```

```

- cantidad: int
- precioUnitario: double
- subtotal: double
}

class "Evento" <<entity>> {
- fechaEvento: Date
- lugar: String
- estadoEvento: String
}

class "Paquete" <<entity>> {
- idPaquete: String
- nombre: String
- descripcion: String
- precioFinal: double
- estado: String
}

class "ServicioProducto" <<entity>> {
- idItem: String
- nombre: String
- tipo: String
- descripcion: String
- precioBase: double
- estado: String
}

class "Insumos" <<entity>> {
- idInsumo: String
- nombre: String
- marca: String
- costo: double
}

' =====
' REPOSITARIOS
' =====

class "RepositorioCotizacion" <<repository>> {
+ guardar(cotizacion: Cotizacion): boolean
+ buscarPorId(id: String): Cotizacion
+ listarPorFiltro(criterio: String): List<Cotizacion>
+ actualizar(cotizacion: Cotizacion): void
}

class "RepositorioCliente" <<repository>> {
+ existeCedulaRuc(cedulaRuc: String): boolean
+ guardar(cliente: Cliente): boolean
+ buscarPorId(id: String): Cliente
+ listarPorFiltro(criterio: String): List<Cliente>
+ actualizar(cliente: Cliente): void
}

class "RepositorioPaquete" <<repository>> {
+ existeNombre(nombre: String): boolean
+ guardar(paquete: Paquete): boolean
+ buscarPorId(id: String): Paquete
+ listarPorFiltro(criterio: String): List<Paquete>
+ actualizar(paquete: Paquete): void
}

class "RepositorioCatalogoInsumos" <<repository>> {
+ existeNombre(nombre: String): boolean
+ guardar(insumo: Insumo): boolean
+ buscarPorId(id: String): Insumo
+ listarPorFiltro(criterio: String): List<Insumo>
+ actualizar(insumo: Insumo): void
+ eliminar(id: String): void
}

' =====
' RELACIONES
' =====

FormularioCotizacion --> ControlCotizacion
FormularioCliente --> ControlCliente
FormularioPaquete --> ControlPaquete
FormularioCatalogoInsumos --> ControlCatalogoInsumos

ControlCotizacion --> RepositorioCotizacion
ControlCotizacion --> RepositorioCliente
ControlCotizacion --> RepositorioPaquete

ControlCliente --> RepositorioCliente
ControlPaquete --> RepositorioPaquete
ControlCatalogoInsumos --> RepositorioCatalogoInsumos

```

```

RepositorioCliente o-- "0..*" Cliente
RepositorioCotizacion o-- "0..*" Cotizacion
RepositorioPaquete o-- "0..*" Paquete
RepositorioCatalogoInsumos o-- "0..*" Insumos

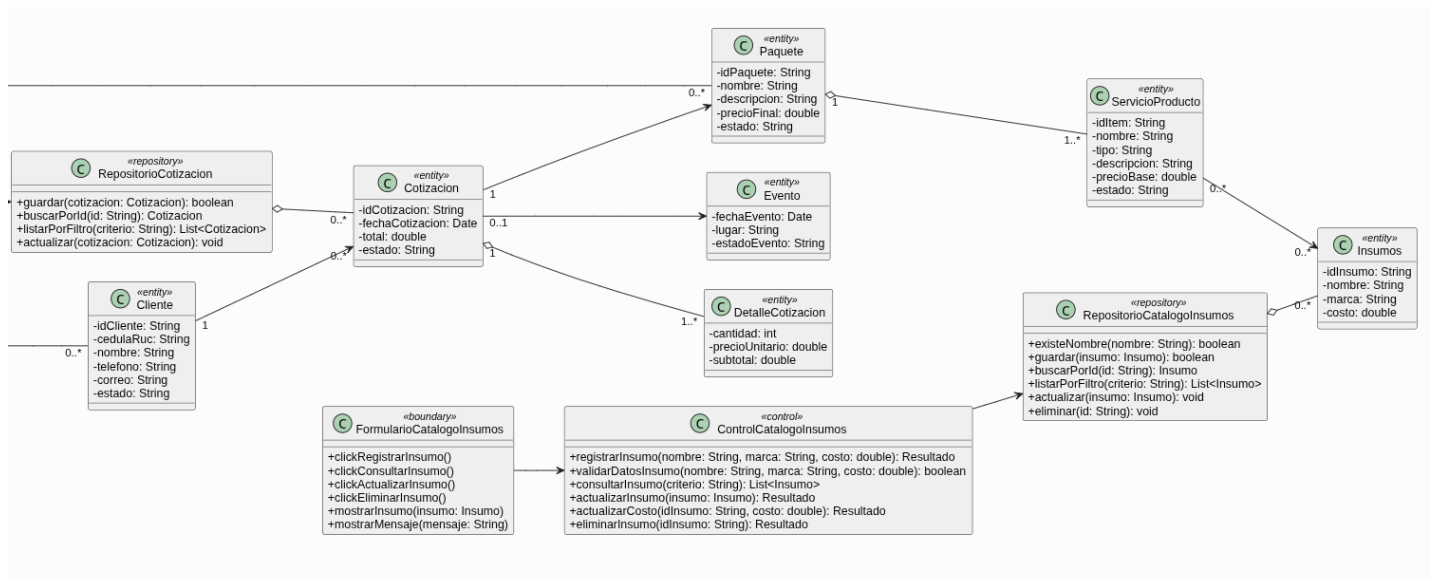
```

```

Cliente "1" --> "0..*" Cotizacion
Cotizacion "1" o-- "1..*" DetalleCotizacion
Cotizacion "0..1" --> Evento
Cotizacion "1" --> Paquete
Paquete "1" o-- "1..*" ServicioProducto
ServicioProducto "0..*" --> "0..*" Insumos

```

@enduml



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Administrar Cotización	Captura datos para generar, consultar o cambiar el estado de una cotización.	Valida cliente, paquete, fecha y coordina el cálculo y registro.	Usa Cotizacion, DetalleCotizacion, Cliente, Evento, Paquete y RepositorioCotizacion.	Secuencia: seleccionar cliente y paquete -> validar datos -> calcular total -> guardar cotización -> confirmar.
Administrar Cliente	Captura y muestra datos de cliente desde el formulario.	Valida campos obligatorios, unicidad de cédula/RUC y coordina el guardado o actualización.	Usa Cliente y RepositorioCliente.	Secuencia: ingresar datos -> validar -> verificar duplicidad -> crear cliente -> guardar -> confirmar.
Administrar Paquetes	Captura datos del paquete y presenta su detalle.	Valida nombre, composición y coordina creación o actualización de paquete.	Usa Paquete, ServicioProducto y RepositorioPaquete.	Secuencia: ingresar paquete -> seleccionar servicios/productos -> validar -> guardar paquete -> confirmar.
Administrar catálogo de Insumos	Captura nombre, marca y costo/precio del insumo.	Valida datos básicos, verifica duplicidad y coordina registro, actualización o eliminación.	Usa Insumo y RepositorioInsumo.	Secuencia: ingresar insumo -> validar -> verificar duplicidad -> guardar insumo -> confirmar.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se elabora un diagrama por cada caso de uso principal seleccionado, evitando diagramas sobrecargados y manteniendo la trazabilidad con las clases identificadas.

8.1 Secuencia: Administrar Cotización - Generar cotización

```
@startuml
title Generar Cotización

actor "Administrador de Cotizaciones" as AC
boundary "FormularioCotizacion" as UI
control "ControlCotizacion" as Ctrl
database "RepositorioCliente" as RepoCli
database "RepositorioPaquete" as RepoPaq
entity "Cotizacion" as Cot
database "RepositorioCotizacion" as RepoCot

AC -> UI : Solicita generar cotización
UI -> Ctrl : generarCotizacion(clienteId, paqueteId, fechaEvento)

Ctrl -> RepoCli : buscarPorId(clienteId)
RepoCli --> Ctrl : cliente encontrado

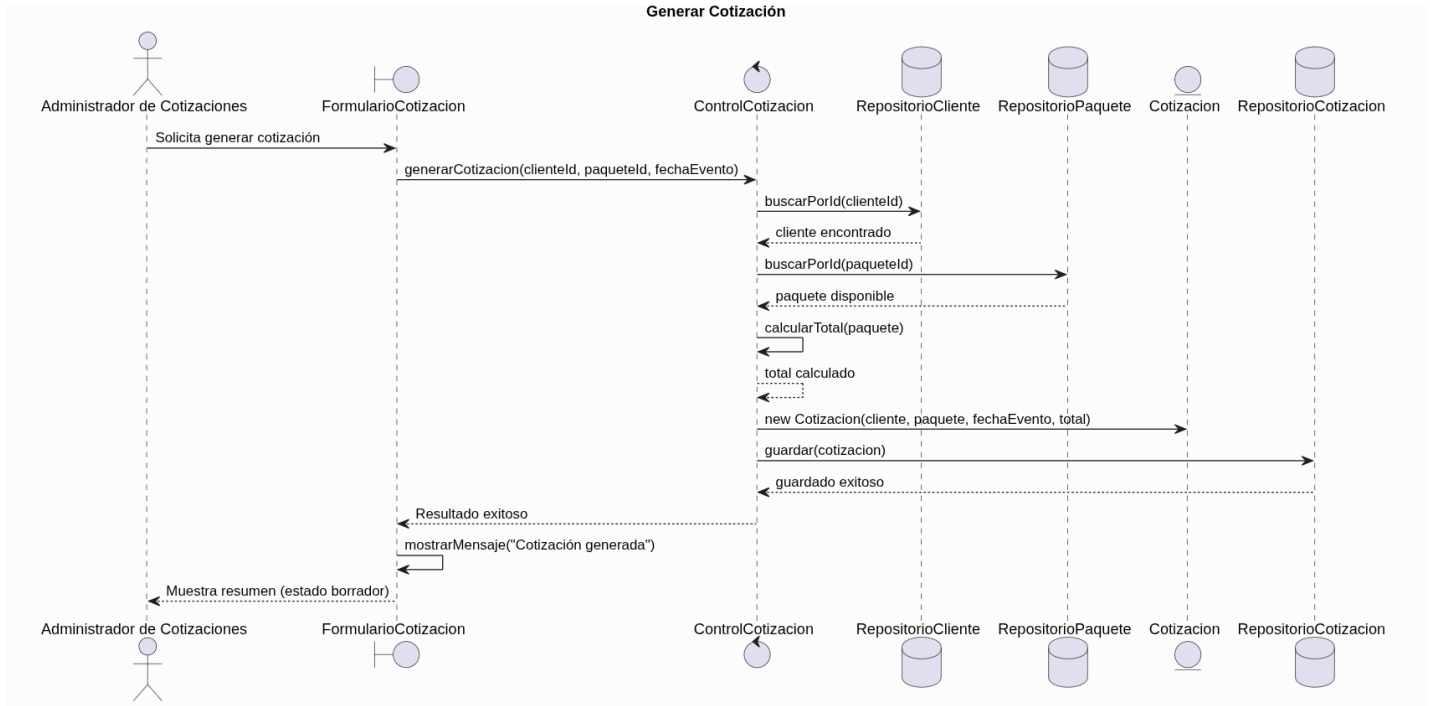
Ctrl -> RepoPaq : buscarPorId(paqueteId)
RepoPaq --> Ctrl : paquete disponible

Ctrl -> Ctrl : calcularTotal(paquete)
Ctrl --> Ctrl : total calculado

Ctrl -> Cot : new Cotizacion(cliente, paquete, fechaEvento, total)

Ctrl -> RepoCot : guardar(cotizacion)
RepoCot --> Ctrl : guardado exitoso

Ctrl --> UI : Resultado exitoso
UI -> UI : mostrarMensaje("Cotización generada")
UI --> AC : Muestra resumen (estado borrador)
@enduml
```



8.2 Secuencia: Administrar Cliente - Registrar cliente

```

@startuml
title Registrar Cliente

actor "Administrador de Cotizaciones" as AC
boundary "FormularioCliente" as UI
control "ControlCliente" as Ctrl
entity "Cliente" as Cli
database "RepositorioCliente" as Repo

AC -> UI : Ingresa datos del cliente
AC -> UI : Clic en Registrar

UI -> Ctrl : registrarCliente(cedulaRuc, nombre, telefono, correo)

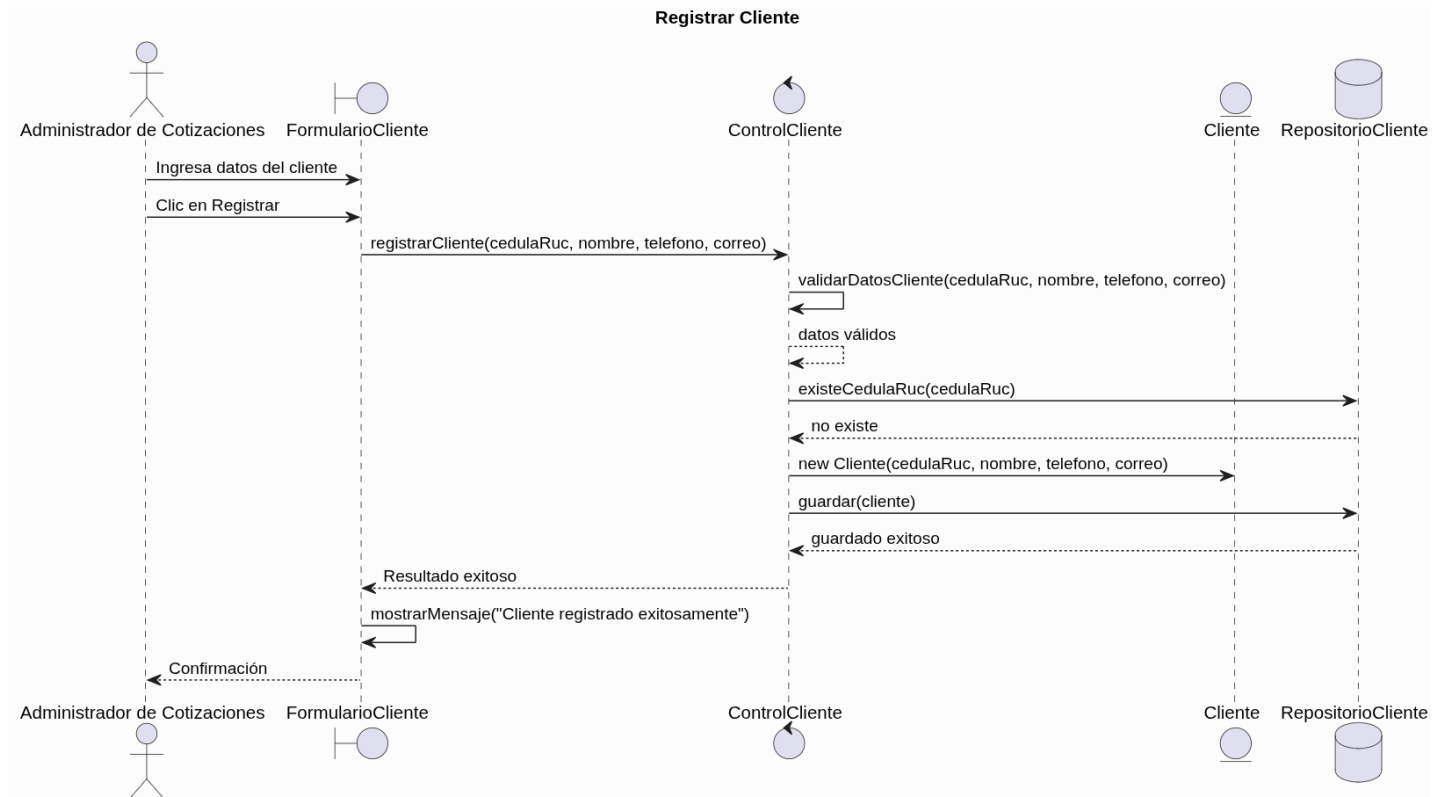
Ctrl -> Ctrl : validarDatosCliente(cedulaRuc, nombre, telefono, correo)
Ctrl --> Ctrl : datos válidos

Ctrl -> Repo : existeCedulaRuc(cedulaRuc)
Repo --> Ctrl : no existe

Ctrl -> Cli : new Cliente(cedulaRuc, nombre, telefono, correo)

Ctrl -> Repo : guardar(cliente)
Repo --> Ctrl : guardado exitoso

Ctrl --> UI : Resultado exitoso
UI -> UI : mostrarMensaje("Cliente registrado exitosamente")
UI --> AC : Confirmación
@enduml
  
```

8.3 Secuencia: Administrar Paquetes - Registrar paquete

```

@startuml
title Registrar Paquete

actor "Administrador del Sistema" as AS
boundary "FormularioPaquete" as UI
control "ControlPaquete" as Ctrl
entity "Paquete" as Paq
database "RepositorioPaquete" as Repo

AS -> UI : Ingresar nombre y descripción
AS -> UI : Selecciona servicios/productos
AS -> UI : Clic en Guardar

UI -> Ctrl : registrarPaquete(nombre, descripcion, items)

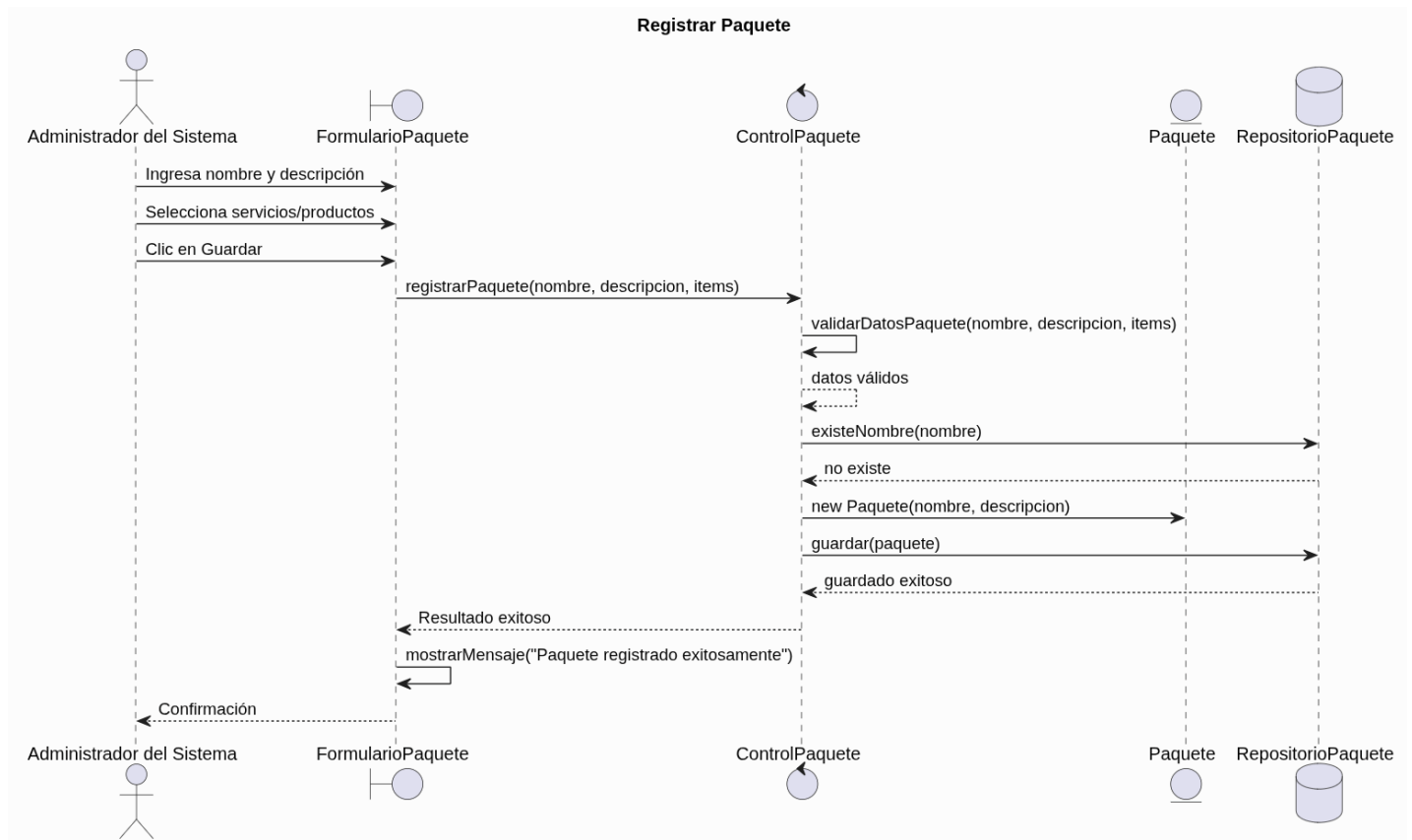
Ctrl -> Ctrl : validarDatosPaquete(nombre, descripcion, items)
Ctrl --> Ctrl : datos válidos

Ctrl -> Repo : existeNombre(nombre)
Repo --> Ctrl : no existe

Ctrl -> Paq : new Paquete(nombre, descripcion)

Ctrl -> Repo : guardar(paquete)
Repo --> Ctrl : guardado exitoso

Ctrl --> UI : Resultado exitoso
UI -> UI : mostrarMensaje("Paquete registrado exitosamente")
UI --> AS : Confirmación
@enduml
  
```



8.4 Secuencia: Administrar catálogo de Insumos - Registrar insumo

@startuml

title Registrar Insumo

actor "Encargado de Inventario" as EI

boundary "FormularioCatalogoInsumos" as UI

control "ControlCatalogoInsumos" as Ctrl

entity "Insumos" as Ins

database "RepositorioCatalogoInsumos" as Repo

EI -> UI : Ingresa datos del insumo

EI -> UI : Clic en Registrar

UI -> Ctrl : registrarInsumo(nombre, marca, costo)

Ctrl -> Ctrl : validarDatosInsumo(nombre, marca, costo)

Ctrl --> Ctrl : datos válidos

Ctrl -> Repo : existeNombre(nombre)

Repo --> Ctrl : no existe

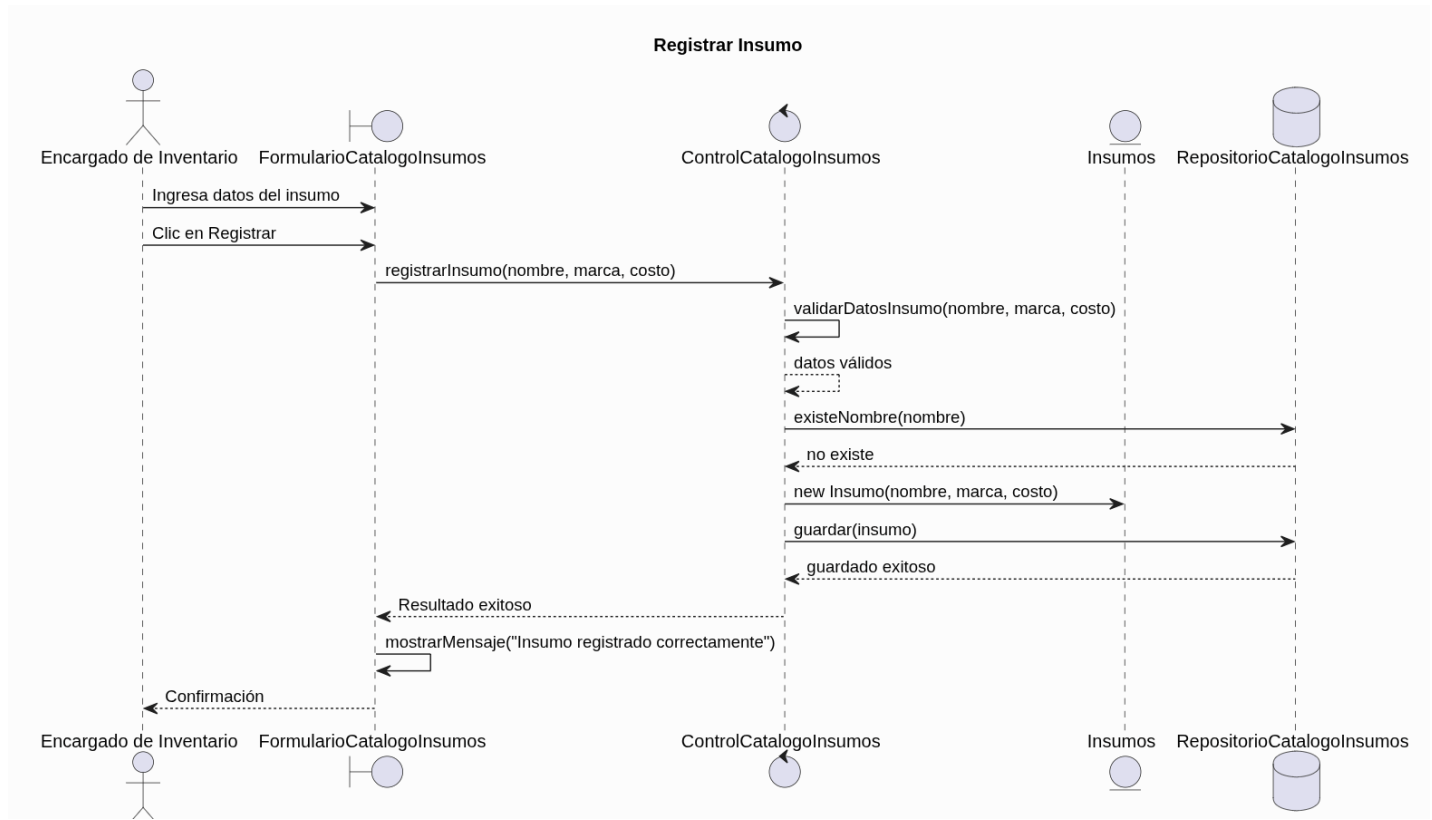
Ctrl -> Ins : new Insumo(nombre, marca, costo)

Ctrl -> Repo : guardar(insumo)

Repo --> Ctrl : guardado exitoso

Ctrl --> UI : Resultado exitoso

UI -> UI : mostrarMensaje("Insumo registrado correctamente")



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Administrar Cotización	Administrador de Cotizaciones autenticado; existen clientes y paquetes activos.	Seleccionar Generar Cotización -> seleccionar cliente y paquete -> ingresar fecha del evento -> calcular total -> guardar cotización.	Cotización registrada en estado Borrador y disponible para consulta o edición.
Administrar Cliente	Administrador de Cotizaciones autenticado; el cliente no debe estar duplicado por cédula/RUC.	Ingresar datos del cliente -> validar campos obligatorios -> verificar duplicidad -> guardar registro.	Cliente registrado y disponible para ser seleccionado en una cotización.
Administrar Paquetes	Administrador del Sistema autenticado; existen servicios/productos disponibles para conformar la oferta.	Ingresar datos del paquete -> seleccionar servicios/productos -> validar datos -> guardar paquete.	Paquete registrado y disponible para ser utilizado en cotizaciones.
Administrar catálogo de Insumos	Encargado de Inventario autenticado; el insumo no debe existir previamente en el catálogo.	Ingresar nombre, marca y costo/precio -> validar datos -> verificar duplicidad -> guardar insumo.	Insumo registrado como referencia básica para el sistema de cotización.

10. Procedimiento recomendado para el estudiante

- Revisar la guía 1 adaptada a FastKote y confirmar los cuatro casos de uso seleccionados.
- Redactar requisitos funcionales de nivel 0 para Administrar Cotización, Administrar Cliente, Administrar Paquetes y Administrar catálogo de Insumos.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones necesarias.
- Derivar clases de análisis: clases frontera para formularios, clases control para coordinar la lógica, entidades del dominio y repositorios de análisis.

- Construir un diagrama de secuencia por cada caso de uso principal seleccionado.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases de análisis.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- Las entidades Cliente, Cotización, Paquete e Insumo conservan los atributos necesarios para el análisis.
- El catálogo de insumos no mezcla funcionalidades fuera del alcance de la propuesta 1, como stock, compras o movimientos de inventario.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actores, sistema, casos de uso y relaciones justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para los cuatro casos de uso seleccionados.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada caso de uso principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia permite comprobar que el modelo de FastKote no se construye como piezas aisladas. Los casos de uso explican qué necesita realizar cada actor dentro del sistema; las clases de análisis muestran qué objetos colaboran para cumplir esas funciones; y las secuencias detallan cómo interactúan dichos objetos durante la ejecución de cada operación principal.

En este trabajo, los casos Administrar Cotización, Administrar Cliente, Administrar Paquetes y Administrar catálogo de Insumos se conectan directamente con los requisitos funcionales de nivel 0 y con las clases de frontera, control, entidad y repositorio propuestas. Esta trazabilidad mejora la claridad del análisis, evita contradicciones entre modelos y prepara

una base más sólida para el diseño posterior del sistema. Además, se mantiene delimitado el alcance del catálogo de insumos como un listado básico de referencia, sin incorporar funcionalidades de inventario que no pertenecen a la propuesta 1.